```
# TABLE-ONLY reconstruction. Section A uses ONLY an exact integer relation table of
24 weights (2r in {-2..2}); T is not touched until Section B.
# A: sets from the table -> support from zero-sum rule -> saturated relation lattice
-> table automorphisms -> invariant sign characters.
# B: only now compare with T (support and character), over many isomorphisms. C:
uniqueness of the invariant tensor under the algebra.
import numpy as np, io, contextlib, itertools, networkx as nx
from networkx.algorithms.isomorphism import GraphMatcher
with contextlib.redirect_stdout(io.StringIO()):
exec(open('39_robustness_of_T_sign_structure.py').read())
# ---------- exact table (doubled integer coordinates; 2r = dot/2)
vec=[]
for i in range(4):
    for s in (2,-2): v=[0]*4; v[i]=s; vec.append(v)
for sg in itertools.product([1,-1],repeat=4):
    if sum(x<0 for x in sg)%2==0: vec.append(list(sg))
for sg in itertools.product([1,-1],repeat=4):
    if sum(x<0 for x in sg)%2==1: vec.append(list(sg))
V=np.array(vec); dot=V@V.T; assert (dot%2==0).all(); t=dot//2
print("A. exact table: 24x24, entries 2r take
values",sorted(set(t.ravel().tolist())))
# ---- A1 sets from the table: same set <=> relation value is an integer (2r even)
g=nx.Graph(); g.add_nodes_from(range(24))
for i in range(24):
    for j in range(i+1,24):
        if t[i,j]%2==0: g.add_edge(i,j)
comps=sorted([sorted(c) for c in nx.connected_components(g)],key=lambda c:c[0]);
print("A1. components of the integer-relation graph:",len(comps),"of sizes",[len(c)
for c in comps])
loc={n:(ci,k) for ci,c in enumerate(comps) for k,n in enumerate(c)}
# ---- A2 support from the zero-sum rule  |a+b+c|^2 = 3 + 2(r_ab+r_ac+r_bc) = 0  <=>
t_ab+t_ac+t_bc = -3
sup=[(loc[a][1],loc[b][1],loc[c][1]) for a in comps[0] for b in comps[1] for c in
comps[2] if t[a,b]+t[a,c]+t[b,c]==-3]
print("A2. one-per-set triples with zero weight-sum:",len(sup))
# ---- A3 incidence, saturated relation lattice
Mt=incidence(sup); Ct,At,rt=int_left_kernel(Mt); print("A3. incidence
rank",np.linalg.matrix_rank(Mt),"| saturated relation lattice rank",len(Ct))
# ---- A4 automorphisms of the exact table
G=nx.complete_graph(24)
for i in range(24):
    for j in range(i+1,24): G[i][j]["t"]=int(t[i,j])
auts=[tuple(m[i] for i in range(24)) for m in GraphMatcher(G,G,edge_match=lambda
a,b:a["t"]==b["t"]).isomorphisms_iter()]
def comp_of(n): return loc[n][0]
def tau_of(p):
```

```
    img=[set(comp_of(p[n]) for n in comps[m]) for m in range(3)]
    return tuple(next(iter(x)) for x in img) if all(len(x)==1 for x in img) else None
pos={e:r for r,e in enumerate(sup)}
def Bmat(p,tau):
    pi={}
    for e in sup:
        new=[None]*3
        for m in range(3): new[tau[m]]=loc[p[comps[m][e[m]]]][1]
        new=tuple(new)
        if new not in pos: return None
        pi[e]=new
    cols=[]
    for c in Ct:
        pc=np.zeros(len(sup),int)
        for e in sup: pc[pos[pi[e]]]=c[pos[e]]
        x=np.rint(np.linalg.lstsq(Ct.T.astype(float),pc.astype(float),rcond=None)
[0]).astype(int)
        if not (x@Ct==pc).all(): return None
        cols.append(x)
    return np.array(cols).T
rows=[];used=0
for p in auts:
    ta=tau_of(p)
    if ta is None: continue
    B=Bmat(p,ta)
    if B is None: continue
    used+=1; rows.extend(((B.T-np.eye(14,dtype=int))%2).tolist())
def f2_nullspace(rows,n=14):
    A=np.array(rows)%2; r=0; piv=[]
    for c in range(n):
        pr=[i for i in range(r,len(A)) if A[i,c]]
        if not pr: continue
        A[[r,pr[0]]]=A[[pr[0],r]]
        for i in range(len(A)):
            if i!=r and A[i,c]: A[i]^=A[r]
        piv.append(c); r+=1
    free=[c for c in range(n) if c not in piv]; basis=[]
    for f in free:
        x=np.zeros(n,int); x[f]=1
        for ri,pc in enumerate(piv):
            if A[ri,f]: x[pc]=1
        basis.append(x)
    return basis
ns=f2_nullspace(rows)
print(f"A4. exact-table automorphisms: {len(auts)}; acting on the support: {used};
invariant sign characters: 2^{len(ns)} = {2**len(ns)}")
x=ns[0]; print("    unique nontrivial invariant character (values on the saturated
basis):",[(-1)**int(v) for v in x])
# ====================== Section B: T is used from here on ======================
print("\nB. comparison with T (first use of T)")
```

```
Wall=np.vstack(W0); tT=np.rint(2*(Wall@Wall.T)).astype(int); print("   T-derived
table equals an integer table 2r:",np.abs(2*(Wall@Wall.T)-tT).max()<1e-9)
GT=nx.complete_graph(24)
for i in range(24):
    for j in range(i+1,24): GT[i][j]["t"]=int(tT[i,j])
isos=[];
for k,m in enumerate(GraphMatcher(G,GT,edge_match=lambda
a,b:a["t"]==b["t"]).isomorphisms_iter()):
    isos.append(m)
    if k>=199: break
print("   isomorphisms exact table -> T-derived table sampled:",len(isos))
idxset=set(idx0); posT={e:r for r,e in enumerate(idx0)}; allsup=True; maxdev=0;
allchar=True; ncheck=0
for m in isos:
    mapped=[]
    for e in sup:
        nodes=sorted(m[comps[k][e[k]]] for k in range(3))        # sort by T-side set
index
        mapped.append(tuple(n%8 for n in nodes))
    if set(mapped)!=idxset or len(set(mapped))!=len(sup): allsup=False; continue
    for j,c in enumerate(Ct):
        val=np.prod([Tw0[mapped[r]]**int(c[r]) for r in range(len(sup)) if c[r]!=0]);
want=(-1)**int(x[j])
        maxdev=max(maxdev,abs(val-want)); ncheck+=1
        if abs(val-want)>1e-9: allchar=False
print("   derived support equals T's support (all sampled isomorphisms):",allsup)
print("   table-derived nontrivial character equals chi_T on all 14 basis vectors,
all sampled isomorphisms:",allchar,"| max deviation %.1e over %d evaluations"%
(maxdev,ncheck))
assert allsup and allchar and maxdev<1e-9
# ======================== Section C: uniqueness of the invariant tensor
=======================
cols=[]
for s in range(3):
    for M in Sb:
        r=[np.einsum('ai,ajk->ijk',M,T0),np.einsum('bj,ibk-
>ijk',M,T0),np.einsum('ck,ijc->ijk',M,T0)][s]; cols.append(r.ravel())
u,sv,vt=np.linalg.svd(np.array(cols).T); null=vt[np.sum(sv>TOL):]
I8=np.eye(8); ops=[]
for v in null:
    A,B,C=trip(v);
ops.append(np.kron(A.T,np.kron(I8,I8))+np.kron(I8,np.kron(B.T,I8))+np.kron(I8,np.kron
(I8,C.T)))
sv2=np.linalg.svd(np.vstack(ops),compute_uv=False)
print("\nC. tensors on 8x8x8 invariant under all %d algebra elements: dimension %d
(singular values: smallest nonzero %.2e, largest dropped %.1e)"%
(len(null),int((sv2<1e-9).sum()),sv2[sv2>=1e-9].min(),sv2[sv2<1e-9].max()))
```